

Internship Final Paper

Christian Bai | RISCPoint Advisory Group | Old Dominion University

Introduction

Getting into RISCPoint started with a personal connection. Someone who knew the team put my name in front of the right people, which got me through the door. Staying there was a different matter. From the first week I made it clear I was there to work and learn, and that effort is what determined how the rest of the internship went.

RISCPoint Advisory Group is a cybersecurity consulting firm founded in 2018 and based in the United States. The company serves both private and public sector clients. Private sector work includes SOC assessments and ISO compliance consulting. Federal-facing work covers FedRAMP authorization support and CMMC assessments. The client base is primarily organizations requiring third-party security advisory, from SaaS companies pursuing cloud authorization to defense contractors working through compliance frameworks. The firm runs lean, which means junior staff get assigned to real client work and interact directly with senior staff without the buffer layers common at larger organizations.

Before the internship started, I set four learning objectives in my Memorandum of Agreement. The first was conducting vulnerability assessments and penetration tests using industry-standard tools and methods. The second was identifying and documenting security vulnerabilities in web applications, networks, and systems. The third was writing penetration testing reports that communicate findings and recommendations clearly to different audiences. The fourth was applying knowledge of attack vectors and defensive measures to improve client security posture. Having those four objectives documented before I started gave the experience a concrete framework and made it easier to measure what I was actually getting out of each engagement as the months progressed.

The beginning of the internship looked nothing like where I ended up. I was onboarded as a trial run on a DevOps team, where my job was writing documentation and running QA tests for a SaaS product that performed automated penetration testing. The work was not difficult, which in some ways made it harder to sit through. Most of my time went toward reviewing tool outputs and writing up results in a prescribed format. There was no actual offensive security work happening, and I was aware early on that this was not where I wanted to spend the rest of the internship.

The shift happened when the project leadership changed. Once a new person took over that engagement, I made a case for moving to the penetration testing side of the firm. That turned out to be the most consequential thing that happened during the entire internship. My supervisor walked me through tools, techniques, and concepts where I had gaps. Early weeks involved a lot of shadowing while I got oriented to how professional engagements were scoped and executed. Over time he handed me more responsibility, and eventually I was running engagements from start to finish on my own, handling everything from initial reconnaissance through final report delivery. I did not expect that level of independence to happen as quickly as it did.

My impressions of the company shifted completely once I moved into pentesting. The DevOps phase had left me lukewarm. After the transfer, that changed. Once you demonstrate you can handle the work, there is no one looking over your shoulder telling you what to test next. You are running live assessments against real client infrastructure, and the stakes of doing that well are tangible in a way that lab work is not. That environment pushed me to perform at a higher level than I would have in a simulated academic context.

Management Environment

The management structure at RISCPoint is about as flat as it gets. The penetration testing department has three people: my supervisor, myself, and an outside contractor. Work gets assigned directly, usually through a Slack message or a quick conversation.

I came in with no real penetration testing experience and had never touched Burp Suite before. My supervisor had to walk me through the basics before I could do much of anything useful. He covered tooling, methodology, how to approach a scope, and how to think about what you are actually looking for during a test. The early check-ins were less about confirming progress and more about making sure I had enough foundation to move forward at all. He was not running me against a rubric. He was making sure I had what I needed, and that hands-on involvement early on was what made the rest of the internship possible.

As I demonstrated I could handle engagements independently, the structure loosened up. Now there is a standing Monday check-in to go over how the week looks, whether I have any blockers, and what the next project is. Outside of that, we communicate through Slack when something comes up. The dynamic shifted from intern-supervisor to something closer to junior colleague. He expects me to use my own judgment, flag issues when they arise, and take ownership of what I produce.

One thing I noticed about how the team operates is that there is a high tolerance for questions, but the expectation is that you have already tried to work through the problem on your own first. That is a different bar than school, where asking questions is always encouraged regardless of whether you

have attempted the material. In a professional context, showing up with a half-formed question signals that you did not put in the effort before escalating. I adjusted to that dynamic relatively quickly, and it pushed me to be more self-sufficient.

Major Work Duties, Assignments, and Projects

My work at RISCPoint split into two distinct phases. The first was the initial DevOps assignment. The second, which made up most of the internship, was the penetration testing work I eventually transitioned into.

During the DevOps phase, my responsibilities were documentation writing and QA testing for a SaaS product that performed automated penetration testing. QA work meant reviewing application outputs for accuracy, checking whether tool findings matched expected behavior, and logging discrepancies. Documentation meant writing clear technical content describing how the application functioned. Neither task required offensive security knowledge, but the documentation work reinforced a habit I later valued: precision in technical writing. That carried directly into report writing once I moved to pentesting.

Once I transitioned to the penetration testing team, the work became significantly more varied and technically demanding. My first solo engagement was a web application penetration test for a client whose application handled multiple user roles and a range of sensitive functionality. Testing involved examining authentication mechanisms, session management, authorization logic, file upload handling, and API behavior. The vulnerabilities I found on that first independent engagement included Information Disclosure through Application Configuration, Stored Cross-Site Scripting, Parameter Tampering, Lack of Authentication on certain endpoints, an out-of-date version of OpenSSL, and an Insecure File Upload vulnerability. Finding all of those on my first unsupervised test was a confidence-building experience. Each finding required me to understand the root cause clearly enough to explain it in a report that a non-technical stakeholder could read and act on.

The second notable engagement involved a web application with ten distinct user roles, which required mapping the full access control model before testing could begin meaningfully. The critical finding was an authorization bypass that let lower-privileged users reach functions reserved for administrators. The application verified that a user was authenticated but did not check whether that user had permission for specific actions. That gap between authentication and authorization is something that gets discussed in textbooks, but seeing it in a production application with real data at stake made the concept concrete in a way that reading about it does not.

That same engagement surfaced an OAuth 2.0 issue as well. The client was using the implicit flow, which has been deprecated in current OAuth security guidance because of how it exposes access

tokens in the browser. Documenting that finding required referencing the current OAuth 2.0 Security Best Current Practice and explaining to the client why what they had implemented, while it may have been acceptable practice some years ago, no longer meets the current bar. Findings like that one sit at the intersection of technical knowledge and ongoing awareness of evolving standards. You cannot find them by running a scanner.

The third major project was a red team assessment. Unlike a penetration test, which tries to identify as many vulnerabilities as possible within a defined scope, the red team engagement simulated a specific adversary pursuing a defined objective. My objective was to breach the client's production environment boundary from an unauthenticated external position. The engagement followed federal compliance guidelines and used the MITRE ATT&CK framework to map every technique I used to documented adversary tactics. I conducted reconnaissance through passive and active methods, designed and executed spear phishing campaigns, performed vulnerability scanning, and identified a critical authentication system flaw that allowed unauthenticated client registration. The social engineering component required me to research the organization's internal communication patterns and craft pretexts believable enough to fool employees with access to sensitive systems. That phase connected directly to my criminology coursework in a way I had not expected. Rational Choice Theory gave me a useful framework for thinking through which social engineering approaches were worth the effort relative to detection risk.

The most technically demanding engagement was the FedRAMP penetration test I conducted for a federal cloud service provider. The client had over twenty externally facing web applications, a multi-tenant SaaS environment, and a social engineering campaign built into the test plan. The phishing campaign targeted approximately thirty employees using a lookalike domain and a fake Google Drive login page built in GoPhish. On the web application side, the client ran on Microsoft IIS with ASP.NET backends, which required me to adjust my testing methodology. IIS handles error responses and session management differently from Apache or Nginx, and working through those differences sharpened my understanding of how framework-specific behavior affects attack surface. The DOM-based XSS vulnerability I found on that engagement was the most technically interesting finding of the internship. The vulnerable component was a help page viewer that read a URL query string with JavaScript and passed it into `document.write()` to build an `iframe`. The developer had tried to sanitize the input by stripping literal colons and their hex-encoded equivalent but had not accounted for the HTML entity form of a colon. The browser decoded that entity back to a colon after the filter ran, which let me inject a JavaScript URI that executed on page load. The vulnerability was compounded by the fact that the same component was shared across five or six applications in the client's suite, so a single finding affected a large portion of the attack surface.

Writing the FedRAMP report was a significant undertaking on its own. The FedRAMP Penetration Test Guidance requires that each mandatory attack vector be addressed individually and that findings be tied back to the relevant vector. Every evidence section needed to be detailed enough to hold up under review by a Third Party Assessment Organization. That standard of documentation is higher than what a typical commercial engagement requires, and meeting it taught me a great deal about what thorough, defensible reporting actually looks like.

Earlier in the internship, I also encountered AI system security testing for the first time. One client had integrated a large language model chatbot into their application, and I was assigned to assess it. I used direct prompt injection techniques, crafting inputs that convinced the model to output its initialization instructions, and successfully extracted the complete system prompt. The prompt contained the behavioral guidelines, restrictions, and capability boundaries the AI had been configured with. That information could be used to circumvent the intended limitations or to design more targeted attacks against the application. I documented the finding as High severity. It was the first time I had tested an AI system in a professional context, and it was a clear signal that this area of the field is going to become increasingly important.

Cybersecurity Skills and Knowledge

Coming into the internship, I had a working foundation in offensive security tools from coursework and self-study. I knew how to use Burp Suite for intercepting and modifying HTTP requests, had familiarity with Nmap for network enumeration, understood the OWASP Top 10 vulnerability categories at a conceptual level, and had used Metasploit in lab environments. I had also worked extensively with Kali Linux through my CYSE 270 coursework, so the tooling environment was not unfamiliar. That baseline was enough to get started. What became clear quickly was that knowing how to use a tool in a lab is a different thing from using it confidently in a live client engagement.

The first major skill gap I closed on the job was learning to distinguish between scanner-identified issues and actual exploitable vulnerabilities. Early on, I would sometimes take Burp Scanner results at face value, only to discover through manual testing that a flagged issue was either a false positive or a properly functioning security control. Running into WAF responses that mimic SQL injection behavior several times over cured me of that habit. Manual validation became a non-negotiable part of how I approach every engagement.

On the IIS and ASP.NET side, almost everything I learned was on the job. ViewState parameters, the ASP.NET page lifecycle, and how IIS surfaces error information are not topics that come up in academic cybersecurity courses at any significant depth. Working through a FedRAMP engagement

on an ASP.NET stack forced me to research those specifics in real time and develop testing approaches appropriate to that environment.

DOM-based XSS was a skill I developed entirely through the internship. I had performed reflected and stored XSS testing before, but the mechanics of DOM-based vulnerabilities are different. The payload never touches the server. The injection point lives entirely in client-side JavaScript. Understanding how JavaScript reads from sources like the URL query string and passes data into sinks like `document.write()` required me to read source code carefully and trace data flow through the application manually. That approach transfers broadly to any client-side security testing work.

GoPhish was a tool I had heard of but had never used in a real campaign. Setting up phishing infrastructure, registering lookalike domains, building credential capture pages, and analyzing delivery and click metrics taught me the operational side of social engineering assessments. Knowing that phishing works is one thing. Building a campaign and watching the metrics in real time is another experience entirely.

The MITRE ATT&CK framework was something I had encountered in coursework at a surface level, but the red team engagement was the first time I used it as a working reference during an actual test. Mapping every technique I employed to the corresponding ATT&CK tactic and technique ID forced me to think about what I was doing in a structured, adversary-representative way rather than ad hoc. It also changed how I write findings in red team reports, because the framework gives both tester and client defender a shared vocabulary for discussing what happened and what to do about it.

AI system testing is a skill area that nobody formally taught me. When I was assigned that engagement, I had to research prompt injection independently, experiment with different approaches, and develop a methodology that did not exist as a formalized checklist anywhere I could find. That experience reinforced something I think is essential in this field: the ability to encounter a completely new attack surface and build a working testing approach from first principles, without waiting for a course to cover it.

The skill my on-the-job experience changed my understanding of most was report writing. I understood before the internship that documentation mattered. What I did not fully appreciate was how much of the value of a penetration test lives in the quality of the written deliverable. A finding that is identified but not communicated clearly is nearly as useless to a client as a finding that was missed entirely. The discipline of writing findings that explain what was discovered, how it was found, what the business impact is, and how to remediate it, in language that a developer, a manager, and a compliance auditor can each act on, is a craft. I am a noticeably better technical writer now than I was when the internship started.

ODU Curriculum Preparation

Four courses from my ODU coursework connected directly to the work I was doing at RISCPoint. CYSE 300 Introduction to Cybersecurity established the foundational concepts that underpin almost everything in this field, including the CIA triad, threat modeling, basic attack categories, and security frameworks. That grounding made it easier to place individual findings within a broader security posture discussion. CS 462 Cybersecurity Fundamentals went deeper on cryptography, network security, and authentication mechanisms. Understanding why MD5 is cryptographically broken was something CS 462 covered, and that knowledge was directly applicable when I found a client using MD5 for password hashing and needed to explain the risk in a client-facing report.

CYSE 301 Cyber Techniques and Operations was the most operationally relevant course. The tooling it covered, including Metasploit, Nmap, Wireshark, and broader vulnerability assessment methodology, overlapped directly with what I was doing in client engagements. CYSE 270 Linux Systems for Cybersecurity was consistently applicable as well, since Kali Linux is the primary operating environment for pentest work. Comfort with the terminal, scripting, file system navigation, and tool configuration in Linux translated directly into daily use.

The course that provided the least practical value in a day-to-day job context was CYSE 200T. It addresses cybersecurity from a broader societal and policy perspective, which is not without value as background, but it did not give me anything I could apply directly during a client engagement. That is not a criticism of the course itself. Policy and societal context matter in this field. But if the question is what prepared me for the actual work, that course sits at the bottom of the list.

Several experiences at the internship reinforced things I had learned in school. The session management vulnerability I found on my first solo engagement, specifically a logout function that did not invalidate the session token server-side, was exactly the kind of scenario covered in application security coursework. Seeing it in production code rather than a lab exercise made the material feel grounded in a way that classroom instruction often does not. The OAuth implicit flow finding connected similarly to material on authentication protocols I had encountered academically.

There were also experiences that revealed concepts I had not yet encountered in school. IIS-specific testing methodology, the FedRAMP Penetration Test Guidance and its mandatory attack vector documentation requirements, DOM-based XSS mechanics, AI system prompt injection, and the operational logistics of running a phishing campaign were all things I learned entirely on the job. The MITRE ATT&CK framework was familiar at a surface level from school, but using it as a practical reference during a live red team engagement is a different level of familiarity than reviewing it in a lecture slide.

My criminology coursework made a connection to the internship that I had not anticipated. When designing the social engineering campaigns for the red team engagement, I found myself drawing on Rational Choice Theory to evaluate which phishing approaches made the most sense. An attacker does a cost-benefit analysis on every tactic: which approach requires the least effort while maximizing the probability of success and minimizing detection risk? That is Rational Choice Theory applied to offensive security, and having a formal framework for that kind of thinking made my campaign design more deliberate and more effective than it would have been otherwise.

MOA Outcomes Review

My first MOA objective was conducting vulnerability assessments and penetration tests using industry-standard tools and methods. This one was met in full. Over the course of the internship I completed multiple full-cycle engagements using Burp Suite Pro, GoPhish, Nmap, and the MITRE ATT&CK framework, among other tools. The assessment types covered web application security, social engineering, network reconnaissance, and adversary simulation, which represents a solid cross-section of what industry-standard methodology looks like in practice.

The second objective was identifying and documenting security vulnerabilities in web applications, networks, and systems. This was also met in full. Across all engagements, I identified and documented vulnerabilities including DOM-based and Stored XSS, authorization bypass, insecure file upload, OAuth implicit flow misuse, session management failures, GraphQL information disclosure, unauthenticated API endpoints, CSRF, and AI system prompt injection. Each finding was documented with technical evidence, reproduction steps, and business impact analysis.

The third objective was writing penetration testing reports that communicate findings and recommendations effectively. This was the objective I spent the most time developing. Early reports required more feedback and revision than later ones. By the time I was producing the FedRAMP report independently, the deliverables were meeting a compliance-grade standard of documentation. The growth in my report writing from the first engagement to the last is probably the most measurable improvement the internship produced.

The fourth objective was applying knowledge of attack vectors and defensive measures to improve organizational security. This one played out across every engagement, but it was most visible during the red team assessment. That project required thinking about attack chains rather than individual vulnerabilities, tracing how initial access leads to persistence, how persistence enables lateral movement, and how lateral movement achieves the final objective. Recommending controls at each stage of that chain is exactly what the objective describes. The FedRAMP engagement was similar in

that it required assessing the client's security posture across an entire application suite rather than finding isolated issues.

Looking at the four objectives together, the one that grew the most over the internship was report writing. It is the objective that felt most abstract before the internship started and the one that became most concrete through repeated practice. Every engagement produced a deliverable that went to a real client, and every one of those deliverables was a direct measure of whether that third objective was being met. The progression from the first report to the last is something I can point to with confidence.

Most Motivating and Exciting Aspects

The most exciting point of the entire internship was my first solo penetration test. Going into that engagement, I had no idea what I would find or whether I would find anything meaningful. The combination of vulnerabilities I ended up discovering across a single real application was surprising. Stored XSS, Parameter Tampering, Lack of Authentication, Insecure File Upload, an out-of-date cryptographic library, and an Information Disclosure issue. It was the first time I had taken a real client application from scope definition through findings documentation with no supervision over my shoulder, and walking out of that engagement with a full findings list and a report I wrote independently confirmed that the skills I had been building were actually functional in a professional context.

The DOM-based XSS finding on the FedRAMP engagement was the most technically satisfying discovery of the internship. Getting to that bypass required careful reading of the JavaScript source, understanding how the browser handles HTML entity decoding relative to filter execution order, and constructing a payload that threaded through two layers of sanitization the developer had implemented. Vulnerabilities that only reveal themselves after that kind of sustained, methodical analysis are the most engaging part of this work for me.

The AI prompt injection finding was motivating for a different reason. It was the first time I had to build a testing methodology from scratch for an attack surface I had not formally studied. There was no checklist to follow, no course that had walked me through the steps. I researched it independently, experimented with different approaches, and eventually extracted a complete system prompt from a production AI application. What mattered more than the result was the process of figuring out how to approach something entirely unfamiliar. That is the part of offensive security that keeps the work interesting.

More broadly, what kept me motivated throughout the internship was the sense of measurable progression. The work was harder at the end than at the beginning, and I was better at it. That does

not always happen in academic settings, where progress gets reduced to a grade on an assignment rather than whether something actually worked in the real world. At RISCPoint, the feedback loop was direct. Either you found the vulnerability or you did not. Either the report was clear enough to act on or it was not. That directness was motivating in a way that classroom evaluations rarely are.

Most Discouraging Aspects

The most discouraging stretch of the internship was the initial DevOps phase. I spent the first portion writing documentation and running periodic QA tests on a tool that did the pentesting for me. The work was not difficult, which in some ways made it worse. There was no challenge to engage with, no technical problem to solve. It was just time passing.

The frustration was not with the company or with anyone on the team. It was more of a mismatch between what I had hoped the role would be and what it actually was at that stage. Looking back, I understand that internship onboarding rarely starts with the most engaging work, and that part of proving you belong somewhere is doing the less glamorous tasks without complaint. Still, the months spent in that phase felt underutilized, and that was hard to sit with.

The turning point came when the project leadership changed and I was able to make the case for moving to the penetration testing team. Once that happened, the frustration went away almost immediately. The lesson I take from that period is that advocating for yourself, making clear what you are capable of and what you want to be working on, is something interns need to be willing to do rather than waiting for someone else to recognize it. The early patience paid off, but only because I eventually pushed for the work I actually wanted.

Most Challenging Aspects

The most persistent challenge throughout the internship was building a testing methodology that worked under real time constraints. In academic labs, the scope is defined, the vulnerability is planted, and the expectation is that you will find it. In a real engagement, none of that is true. The scope can be broad, the vulnerabilities may or may not exist, and you have a fixed number of hours to cover as much meaningful attack surface as possible. Learning to prioritize, to develop intuition for where vulnerabilities are most likely to hide and when to cut a line of investigation that is not producing results, took time and was a real struggle early on.

The IIS and ASP.NET testing environment on the FedRAMP engagement was technically challenging in a specific way. My prior experience had been almost entirely with Apache and Nginx-backed applications. IIS handles error responses, session management, and authentication differently

enough that my existing mental model did not fully transfer. The ViewState parameter in ASP.NET was one concrete example. Understanding how ViewState works, and what happens when you modify it and observe the server response, required active research and experimentation during the engagement itself. That situation is not unusual in penetration testing, but it highlighted how much of the field is framework-specific knowledge you have to accumulate through exposure rather than instruction.

FedRAMP reporting was the most demanding documentation challenge of the internship. The format requires that every mandatory attack vector specified in the FedRAMP Penetration Test Guidance be addressed individually, with findings tied back to the relevant vector. Evidence sections need to be thorough enough to withstand scrutiny from a Third Party Assessment Organization reviewer who was not present during testing. Writing to that standard independently on a first FedRAMP engagement required significant time and multiple revision passes. Getting comfortable with that format was hard, but the discipline it imposed on my documentation habits made me a better writer for every engagement after it.

Maintaining detailed notes during testing was also a challenge I did not fully anticipate. Early on, I would sometimes rely on memory to reconstruct what I had done during a test, only to find that important details had gone fuzzy by the time I sat down to write the report. Timestamps, command outputs, exact request and response sequences, and screenshots I thought I would remember clearly enough were often not as crisp a few days later. I developed a more systematic note-taking habit over time, but the early engagements where my documentation fell short taught me that lesson the hard way.

There was also a learning curve around scoping judgment that I had not anticipated. Early in the internship, I would sometimes spend too long on low-probability attack paths while missing more obvious issues elsewhere. Part of that was inexperience, but part of it was a lack of comfort cutting my losses on a line of investigation that was not producing anything. Knowing when to move on versus when to keep digging requires a feel for the work that only develops through repeated exposure to real engagements. I do not think there is a shortcut to it other than logging enough hours on actual tests.

Recommendations for Future Interns

The most important thing a future intern can do before starting is develop a real understanding of how HTTP works. That means at the protocol level, down past the surface. How requests are structured, what each header field does and why it matters, how cookies are set and scoped, how

redirects work, how different status codes map to different server behaviors. This is the foundation everything else in web application security testing rests on, and gaps here slow everything down.

Related to HTTP, having working familiarity with major web application frameworks before starting saves a lot of ramp-up time. Knowing that an ASP.NET application behaves differently from a Rails application, and understanding at a rough level why, means that when you encounter a new framework in the field you have a mental model to build on. You do not need to be a developer. You need enough exposure to recognize framework-specific patterns when you see them and know which areas are worth probing more carefully.

Burp Suite Pro is the primary tool for web application testing at this internship and at most firms doing similar work. Being fast and confident with it before starting will let you spend your cognitive resources on the actual testing rather than on figuring out how the tool works. Focus specifically on the Repeater and Intruder modules, practice reading and modifying complex HTTP requests, and practice interpreting response differences. All of that is buildable in a home lab before the internship begins.

Future interns should also expect that the early portion of the experience may not match what they hoped for. The initial onboarding phase may involve less exciting work while trust is being established and the team figures out what you are capable of. Being patient through that period is worth it. The more engaging work comes once the people around you have seen that you are reliable. Do not read a slow start as a signal about the rest of the experience.

Finally, invest in your writing before you arrive. Technical skill gets findings identified. Writing skill is what actually gets them fixed. A penetration test report is the client deliverable, and the quality of that document is how the client judges the quality of the engagement. Being able to write a finding that is clear, specific, actionable, and scoped appropriately for different readers is a skill that separates testers who find vulnerabilities from testers who create lasting value for their clients.

Conclusion

The internship at RISCPoint changed the shape of my last year at ODU in a specific and concrete way. Before it started, I had an academic understanding of penetration testing and a general interest in offensive security. After it, I have a working professional identity in this field. I ran real engagements, wrote reports that went to actual client stakeholders, and found vulnerabilities in production systems that would have represented real risk if left unaddressed. That transition from student to practicing professional happened faster than I expected, and it is the most significant thing the internship produced.

One thing the internship made clear is how much of professional competence is situational and earned rather than transferable directly from a classroom. School teaches principles, and those principles matter. But the moment you sit in front of a live application with a real scope and a real deadline, the principles have to become instincts. That conversion from conscious knowledge to practical instinct only happens through doing the work. The internship gave me enough hours of real testing to start making that conversion in a meaningful way.

The experience also confirmed something about the interdisciplinary approach I have been building at ODU. Cybersecurity, criminology, and law are not separate academic tracks running in parallel. They form a single coherent perspective on a field that touches all three domains. Understanding attacker motivation through criminological frameworks made me more effective on the red team engagement. Understanding compliance frameworks like FedRAMP and GDPR made my reports more useful to clients navigating regulatory requirements. That integration was not obvious to me in a classroom setting. It became obvious when the work made it tangible.

For the remainder of my time at ODU, the internship has given me a clearer sense of where to invest my effort. Advanced exploitation techniques, cloud security architectures, and programming for automation are three areas where my academic exposure has been limited and where I plan to do focused work. The OSCP is on my certification roadmap as well. The internship confirmed that I am at a point where that level of assessment is a realistic next step rather than an aspirational one.

The conclusion of this internship is not really a conclusion. I am planning to move into a full-time Penetration Tester role with RISCPoint in June, before I graduate. The work I have been doing as an intern will continue directly into full-time professional practice. What this internship gave me, beyond skills and knowledge, was confirmation that I can perform effectively in this field and that I want to keep doing this work. Going full-time before graduation was always the plan. Now it is actually happening.